



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Universidad Técnica Federico Santa María

Departamento de Industrias

Laboratorio 3

Automatización de procesos con n8n

ICN292 — Sistemas de Información para la Gestión

Segundo Semestre 2026

Estudiante: Benjamín Lara

RUT: 21.378.238-K

Parámetros: $S = 238$, $U = \$68.000$, $D = 21$ días

Campus: Vitacura

Profesores: José Miguel González Pau
José Luis Sáez Tamayo

Repositorio GitHub:

<https://github.com/benjaminlaraorellana2-creator/ICN292-Lab3-Lara-Benjamin.git>
Septiembre de 2026

Resumen ejecutivo

Este laboratorio desarrolla un producto mínimo viable en n8n para automatizar el triage de solicitudes de devolución de AndesHogar SpA. El flujo reemplaza una revisión manual inicial por un proceso reproducible que recibe solicitudes mediante un Webhook, aplica reglas de negocio en orden de precedencia, registra la decisión, consulta el valor de la UF mediante una API pública, genera una notificación y responde al emisor. Se implementaron además un workflow emisor para procesar las 15 solicitudes entregadas y un workflow programado para consolidar un resumen diario.

Los parámetros personales se calcularon a partir del RUT 21.378.238-K. Se obtuvo $S = 238$, un umbral de monto de $U = \$68.000$ y un plazo máximo de $D = 21$ días. La política se implementó respetando la precedencia: primero datos inválidos, luego rechazo, después revisión y finalmente aprobación. En la ejecución de las 15 solicitudes se obtuvieron 4 aprobaciones, 7 revisiones y 4 rechazos, con un monto total procesado de \$1.251.820 y una tasa de aprobación de 26,67 %.

La solución incorpora trazabilidad mediante una Data Table con identificador de solicitud, SKU, monto, ruta, motivo, parámetros U y D , valor de la UF y monto expresado en UF. El workflow diario consulta esa tabla, filtra por fecha, agrupa por ruta con el nodo Summarize y genera un único mensaje con cantidades, montos y tasa de aprobación. Para un día sin solicitudes se diseñó una rama alternativa que informa cero operaciones y evita divisiones por cero.

También se evaluaron errores y casos borde. Se probaron identificadores vacíos, monto igual a cero, un monto no numérico, los límites $monto = U$ y $dias = D$, una comparación deliberadamente tratada como texto y un SKU inexistente. El caso “veinte mil” provoca una falla de conversión numérica en el Switch, lo que permitió validar la configuración *Stop Workflow* y un *Error Workflow* independiente. Como conclusión, el MVP cumple la función de triage, pero para convertirse en un sistema definitivo debería agregar validación estructural previa, verificación de SKU contra catálogo, un mecanismo real de notificación y controles operacionales adicionales.

Índice

Resumen ejecutivo	1
1. Contexto y parámetros	3
2. Diseño general de la solución	3
2.1. URLs del Webhook	4
3. Parte A — Flujo de triage	4
3.1. Webhook	4
3.2. Switch y reglas de negocio	4
3.3. Consulta de UF y monto en UF	4
3.4. Registro y notificación	5
3.5. Ejecución exitosa del flujo	5
3.6. Resultados de las 15 solicitudes	6
4. Parte B — Flujo programado de resumen	7
4.1. Día sin solicitudes	8
5. Parte C — Errores y casos borde	9
5.1. Tres entradas inválidas	9
5.2. Opciones de On Error	9
5.3. Error Workflow	10
5.4. Ejecución fallida: monto no numérico	11
5.5. Casos límite	12
5.6. Error de tipo a propósito	13
5.7. Casos que el flujo no resuelve	13
6. Conclusiones y recomendación	14
7. Repositorio y reproducción	14
8. Evidencias incluidas	15

1. Contexto y parámetros

El proceso modelado corresponde al triage inicial de solicitudes de devolución recibidas por el canal de e-commerce de AndesHogar SpA. El objetivo del MVP es decidir de forma automática si cada solicitud debe clasificarse como datos inválidos, rechazo, revisión o aprobación, manteniendo la inspección física fuera del alcance del laboratorio.

Para este trabajo se utilizaron los tres últimos dígitos del RUT sin dígito verificador:

$$S = 238$$

El umbral de monto se definió como:

$$U = 30000 + 1000(S \bmod 50)$$

Como $238 \bmod 50 = 38$:

$$U = 30000 + 38000 = \$68.000$$

El plazo máximo se definió como:

$$D = 7 + 7(S \bmod 4)$$

Como $238 \bmod 4 = 2$:

$$D = 7 + 14 = 21 \text{ días}$$

Por tanto, los parámetros utilizados en todos los nodos del flujo fueron:

Parámetro	Valor	Uso
S	238	Semilla personal
U	\$68.000	Umbral de monto
D	21 días	Plazo máximo

2. Diseño general de la solución

Se construyeron tres workflows principales:

1. **Triage:** recibe una solicitud mediante Webhook, aplica las reglas de negocio, consulta la UF, registra la solicitud, prepara una notificación y responde al emisor.
2. **Emisor:** contiene las 15 solicitudes, las separa en ítems individuales y las envía por POST al Webhook del workflow de triage.
3. **Resumen diario:** se ejecuta mediante Schedule Trigger, consulta el registro, filtra las solicitudes del día, resume por ruta y genera un único mensaje.

Además, para la sección de errores se creó un workflow de prueba con Error Trigger, utilizado únicamente para demostrar la captura de fallas. Este flujo no reemplaza ninguno de los tres workflows principales requeridos para la entrega.

2.1. URLs del Webhook

n8n entrega dos URLs distintas:

- **Test URL:** <https://benjalara.app.n8n.cloud/webhook-test/devoluciones-andeshogar>
- **Production URL:** <https://benjalara.app.n8n.cloud/webhook/devoluciones-andeshogar>

La Test URL se utiliza mientras el workflow está escuchando eventos de prueba desde el editor. La Production URL se utiliza con el workflow publicado y permite recibir solicitudes sin mantener una ejecución manual abierta. Esta diferencia también fue utilizada para validar el Error Workflow, ya que la falla debía producirse en una ejecución automática/publicada.

3. Parte A — Flujo de triage

3.1. Webhook

El flujo se inicia con un Webhook configurado con método POST. El cuerpo recibido contiene los campos `id_solicitud`, `sku`, `unidades`, `monto`, `dias_desde_compra`, `estado_producto` y `email_cliente`. Todos los campos llegan como texto, por lo que las comparaciones numéricas se convierten explícitamente mediante `.toNumber()`.

3.2. Switch y reglas de negocio

El nodo Switch se configuró en modo *Rules*. La primera regla que se cumple determina la ruta y detiene la evaluación de las siguientes condiciones, de modo que se respeta la precedencia definida por la política.

#	Condición	Salida	Interpretación
1	<code>id_solicitud</code> vacío	INVALIDO_CAMPO	Falta identificador
2	<code>monto.toNumber() <= 0</code>	INVALIDO_MONTO	Monto no positivo
3	<code>dias.toNumber() > 21</code>	RECHAZO_PLAZO	Fuera del plazo
4	<code>estado = danado_por_uso</code>	RECHAZO_ESTADO	Dañado por uso
5	<code>monto.toNumber() > 68000</code>	REVISION_MONTO	Sobre umbral
6	<code>estado = con_fallas</code>	REVISION_FALLA	Producto con fallas
7	Fallback	APROBACION	Ninguna regla anterior

Las siete salidas se consolidaron en cuatro nodos Edit Fields: *Datos inválidos*, *Rechazo*, *Revisión* y *Aprobación*. Cada uno agrega los campos `ruta` y `motivo` manteniendo los datos originales.

3.3. Consulta de UF y monto en UF

Después de consolidar las ramas, el flujo realiza un HTTP Request GET a:

<https://mindicador.cl/api/uf>

La respuesta se combina con los datos originales mediante un nodo Merge. El valor utilizado corresponde a `serie[0].valor`. Luego se calcula:

$$\text{monto_uf} = \frac{\text{monto}}{\text{valor_uf}}$$

En las ejecuciones de evidencia se observó un valor de UF de 40.983,58, variable según la fecha de ejecución.

3.4. Registro y notificación

Cada solicitud se inserta en la Data Table `registro_devoluciones`. Se registran como mínimo:

- fecha,
- identificador,
- SKU,
- monto,
- días desde la compra,
- estado del producto,
- email,
- ruta y motivo,
- *U* y *D*,
- valor de UF y monto en UF.

Luego se genera una notificación estructurada con destinatario, asunto y mensaje. Dado que las direcciones suministradas son direcciones de ejemplo, el MVP prepara la notificación sin depender de credenciales de correo reales.

3.5. Ejecución exitosa del flujo

La Figura 1 muestra una ejecución completa del workflow de triage. En este ejemplo, la solicitud se dirige por la rama de datos inválidos y continúa correctamente hasta el registro, notificación y respuesta.

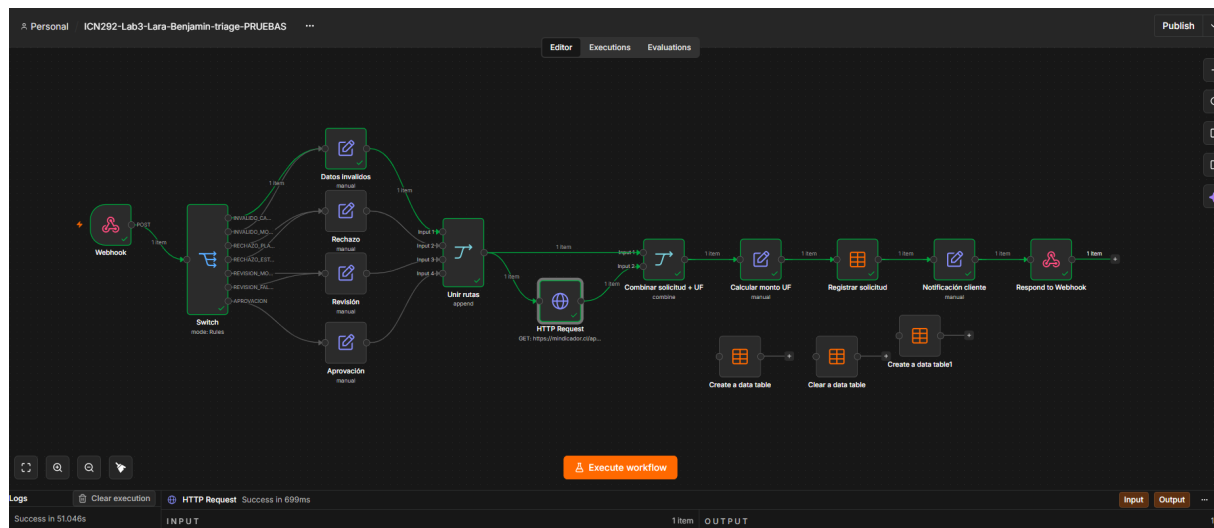


Figura 1: Ejecución exitosa del workflow de triage con ruta de datos inválidos.

3.6. Resultados de las 15 solicitudes

Los resultados obtenidos fueron los siguientes:

Solicitud	Monto	Ruta	Motivo
DEV-1115	\$19.990	APROBACION	APROBACION
DEV-1116	\$69.980	REVISION	REVISION_MONTO
DEV-1117	\$24.990	REVISION	REVISION_FALLA
DEV-1118	\$79.990	REVISION	REVISION_MONTO
DEV-1119	\$45.990	RECHAZO	RECHAZO_PLAZO
DEV-1120	\$89.990	REVISION	REVISION_MONTO
DEV-1121	\$289.990	REVISION	REVISION_MONTO
DEV-1122	\$45.990	APROBACION	APROBACION
DEV-1123	\$39.980	RECHAZO	RECHAZO_ESTADO
DEV-1124	\$49.980	APROBACION	APROBACION
DEV-1125	\$34.990	RECHAZO	RECHAZO_PLAZO
DEV-1126	\$79.990	REVISION	REVISION_MONTO
DEV-C01	\$9.990	APROBACION	APROBACION
DEV-C02	\$289.990	REVISION	REVISION_MONTO
DEV-C03	\$79.990	RECHAZO	RECHAZO_PLAZO

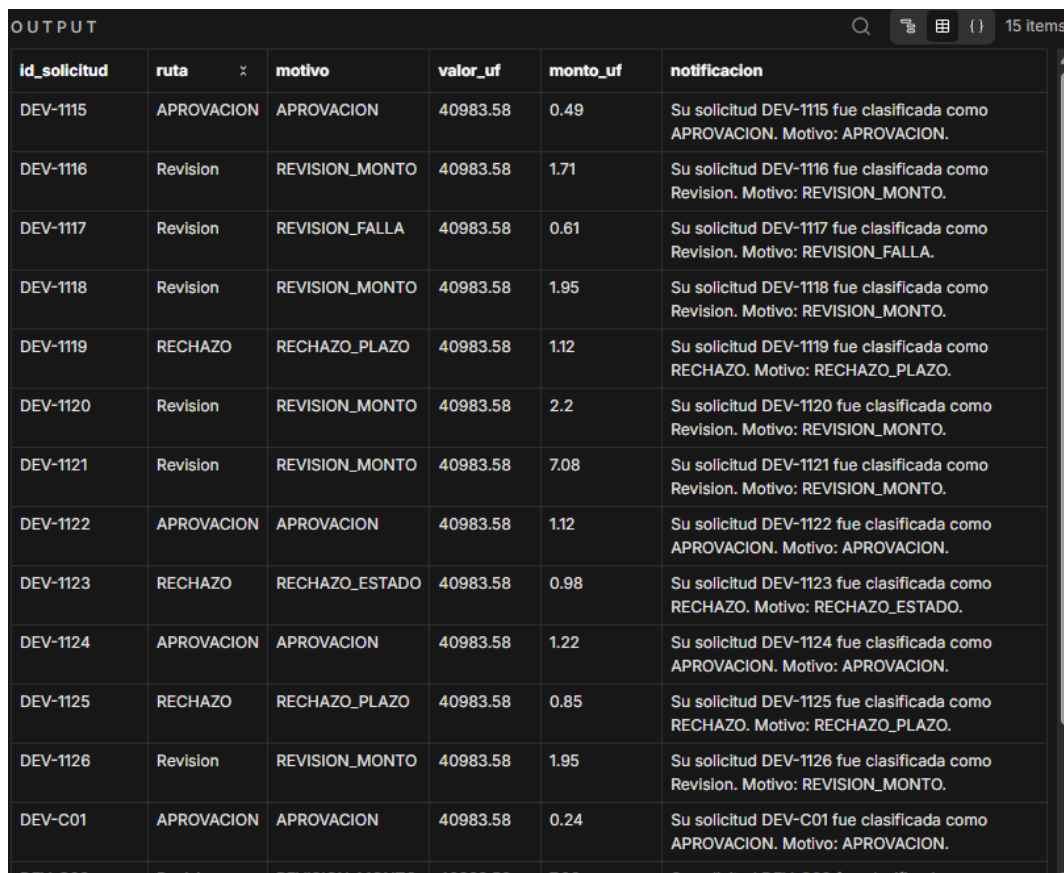
La precedencia también se verifica en casos con más de una causal. Por ejemplo, DEV-1119 está fuera de plazo y además tiene estado **danado_por_uso**; al aparecer primero la regla de plazo, queda registrado como RECHAZO_PLAZO.

El resumen agregado fue:

Ruta	Cantidad	Monto total
APROBACION	4	\$125.950
REVISION	7	\$924.920
RECHAZO	4	\$200.950
TOTAL	15	\$1.251.820

La tasa de aprobación corresponde a:

$$\frac{4}{15} \times 100 = \boxed{26,67\%}$$



id_solicitud	ruta	motivo	valor_uf	monto_uf	notificacion
DEV-1115	APROBACION	APROBACION	40983.58	0.49	Su solicitud DEV-1115 fue clasificada como APROBACION. Motivo: APROBACION.
DEV-1116	Revision	REVISION_MONTO	40983.58	1.71	Su solicitud DEV-1116 fue clasificada como Revision. Motivo: REVISION_MONTO.
DEV-1117	Revision	REVISION_FALLA	40983.58	0.61	Su solicitud DEV-1117 fue clasificada como Revision. Motivo: REVISION_FALLA.
DEV-1118	Revision	REVISION_MONTO	40983.58	1.95	Su solicitud DEV-1118 fue clasificada como Revision. Motivo: REVISION_MONTO.
DEV-1119	RECHAZO	RECHAZO_PLAZO	40983.58	1.12	Su solicitud DEV-1119 fue clasificada como RECHAZO. Motivo: RECHAZO_PLAZO.
DEV-1120	Revision	REVISION_MONTO	40983.58	2.2	Su solicitud DEV-1120 fue clasificada como Revision. Motivo: REVISION_MONTO.
DEV-1121	Revision	REVISION_MONTO	40983.58	7.08	Su solicitud DEV-1121 fue clasificada como Revision. Motivo: REVISION_MONTO.
DEV-1122	APROBACION	APROBACION	40983.58	1.12	Su solicitud DEV-1122 fue clasificada como APROBACION. Motivo: APROBACION.
DEV-1123	RECHAZO	RECHAZO_ESTADO	40983.58	0.98	Su solicitud DEV-1123 fue clasificada como RECHAZO. Motivo: RECHAZO_ESTADO.
DEV-1124	APROBACION	APROBACION	40983.58	1.22	Su solicitud DEV-1124 fue clasificada como APROBACION. Motivo: APROBACION.
DEV-1125	RECHAZO	RECHAZO_PLAZO	40983.58	0.85	Su solicitud DEV-1125 fue clasificada como RECHAZO. Motivo: RECHAZO_PLAZO.
DEV-1126	Revision	REVISION_MONTO	40983.58	1.95	Su solicitud DEV-1126 fue clasificada como Revision. Motivo: REVISION_MONTO.
DEV-C01	APROBACION	APROBACION	40983.58	0.24	Su solicitud DEV-C01 fue clasificada como APROBACION. Motivo: APROBACION.

Figura 2: Salida del workflow emisor con 15 ítems procesados.

4. Parte B — Flujo programado de resumen

El tercer workflow utiliza un Schedule Trigger para ejecutar el proceso una vez al día. La lógica implementada es:

Schedule Trigger → Get Rows → Filter → IF → Summarize → Aggregate → Mensaje

El nodo *Get row(s)* obtiene los registros de `registro_devoluciones`. El filtro compara la fecha ISO almacenada con la fecha del día actual. Si existen solicitudes, el nodo Summarize agrupa por `ruta`, cuenta `id_solicitud` y suma `monto`. Posteriormente, Aggregate reúne las tres filas en un solo ítem y un Edit Fields genera el mensaje consolidado.

La frecuencia escogida fue una ejecución diaria al final de la jornada, a las 23:00. Esto permite resumir la actividad del día en una sola salida.

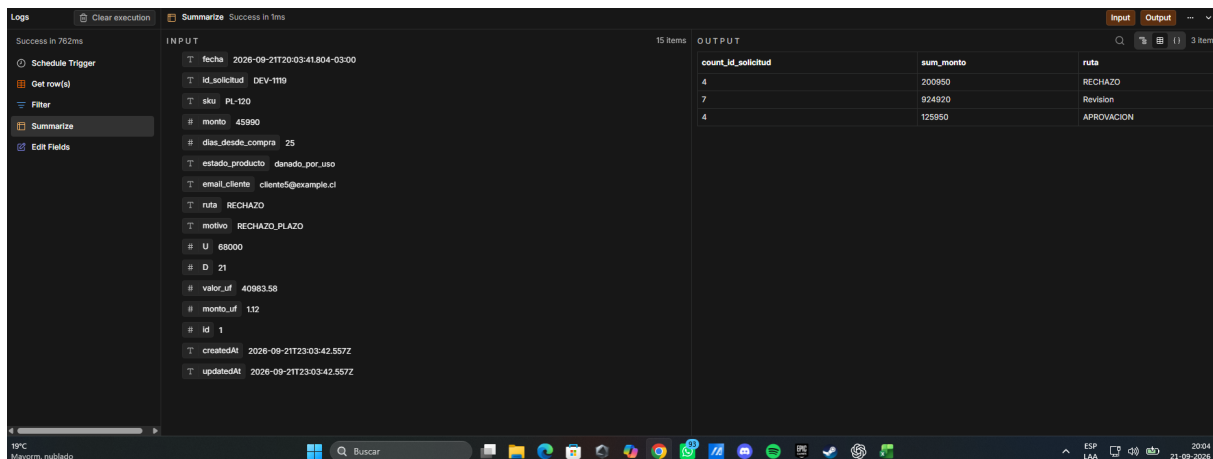


Figura 3: Resultado del nodo Summarize: 4 rechazos, 7 revisiones y 4 aprobaciones.

El mensaje consolidado contiene:

- cantidad de solicitudes por ruta,
- monto total por ruta,
- total de solicitudes del día,
- monto total del día,
- tasa de aprobación.

Para las 15 solicitudes, el mensaje queda conceptualmente:

RESUMEN DIARIO DE DEVOLUCIONES

APROBACION: 4 solicitudes - \$125.950
 REVISION: 7 solicitudes - \$924.920
 RECHAZO: 4 solicitudes - \$200.950

TOTAL SOLICITUDES: 15
 MONTO TOTAL: \$1.251.820
 TASA DE APROBACION: 26,67%

4.1. Día sin solicitudes

Si el filtro no encuentra solicitudes, se utiliza *Always Output Data* y un nodo IF para dirigir la ejecución a una rama alternativa. De esta manera el workflow no termina sin salida y genera un mensaje equivalente a:

RESUMEN DIARIO DE DEVOLUCIONES

No se registraron solicitudes durante el día.
 TOTAL SOLICITUDES: 0
 MONTO TOTAL: \$0
 TASA DE APROBACION: N/A

La tasa se informa como no aplicable para evitar una división 0/0.

5. Parte C — Errores y casos borde

5.1. Tres entradas inválidas

Se evaluaron tres entradas distintas.

Entrada	Resultado esperado	Resultado observado
<code>id_solicitud =</code>	Datos inválidos	DATOS_INVALIDOS / INVALIDO_CAMPO
<code>monto = "0"</code>	Datos inválidos	DATOS_INVALIDOS / INVALIDO_MONTO
<code>monto = "veinte mil"</code>	Falla de conversión	Error en Switch: cannot convert to number

La Figura 4 muestra la segunda entrada inválida.

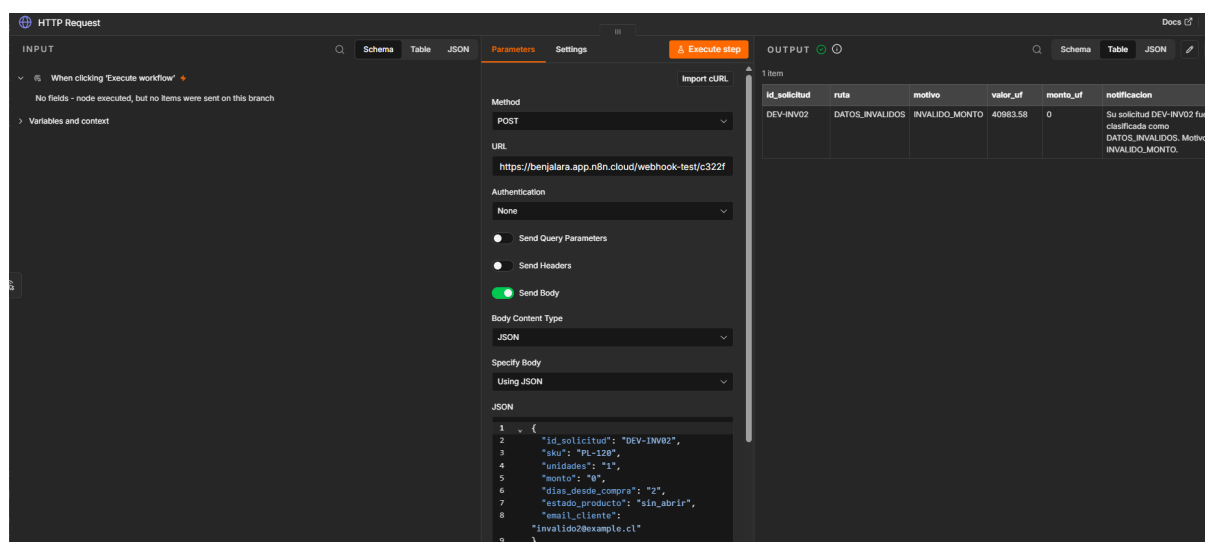


Figura 4: Entrada con monto igual a cero clasificada como INVALIDO_MONTO.

5.2. Opciones de On Error

El nodo Switch ofrece tres opciones de manejo de errores:

- **Stop Workflow:** detiene la ejecución y marca el workflow como fallido.
- **Continue:** entrega el error por la salida normal y permite que el flujo continúe.
- **Continue (using error output):** entrega el error por una salida especial para tratarlo en otra rama.

Para este laboratorio se escogió **Stop Workflow** en el Switch. Un monto no convertible no debe continuar hacia registro ni aprobación, porque podría generar una decisión inválida.

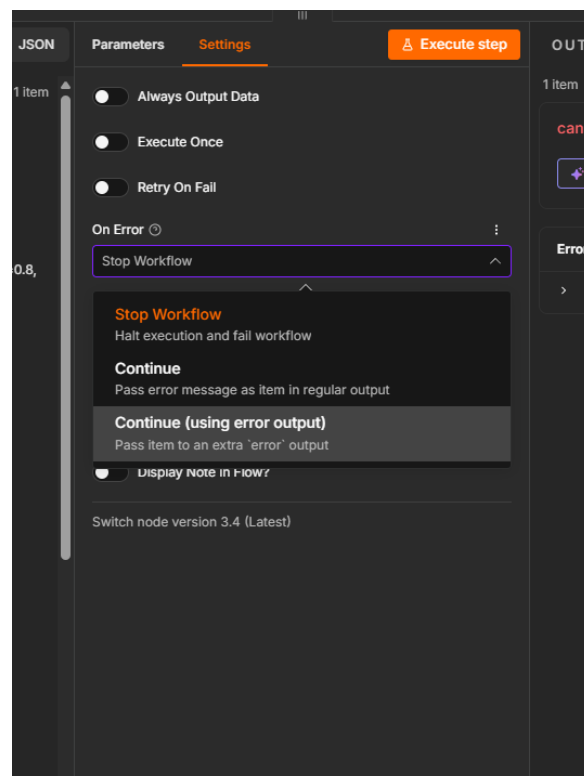


Figura 5: Opciones de manejo de error disponibles en el nodo Switch.

5.3. Error Workflow

Se configuró un workflow independiente iniciado por Error Trigger y conectado a un Edit Fields denominado *Registrar error*. Este flujo registra la fecha, nombre del workflow, mensaje del error y último nodo ejecutado. Luego se seleccionó como Error Workflow del workflow de triage de pruebas.

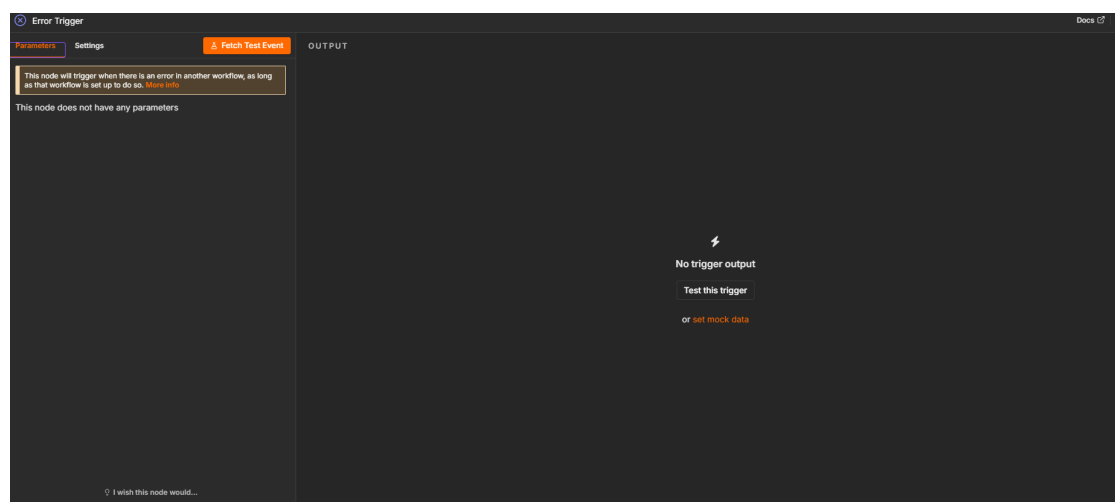


Figura 6: Nodo Error Trigger utilizado para capturar fallas de otro workflow.

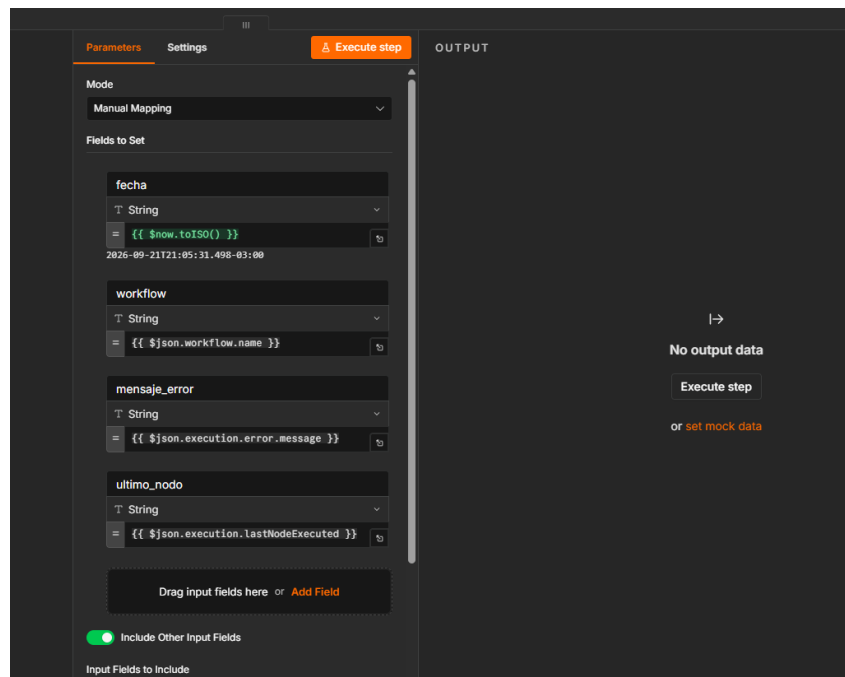


Figura 7: Campos registrados por el Error Workflow.

Un workflow programado puede ejecutarse cuando no existe un usuario observando el editor. Si falla y no tiene un Error Workflow asociado, el error puede quedar únicamente en el historial de ejecuciones, sin desencadenar una acción de tratamiento o notificación. El Error Workflow permite convertir esa falla en un evento procesable.

5.4. Ejecución fallida: monto no numérico

Se envió una solicitud con:

```
"monto": "veinte mil"
```

El Switch intenta convertir el monto mediante `.toNumber()`. Como la cadena no representa un número, el nodo falla con el mensaje `cannot convert to number`, y el flujo se detiene.

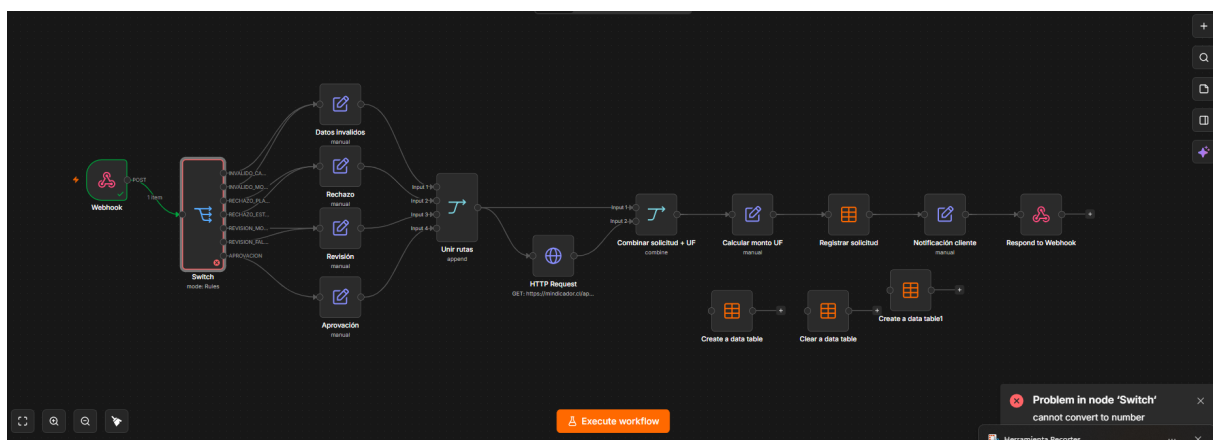


Figura 8: Ejecución fallida al intentar convertir “veinte mil” a número.

Para un sistema definitivo se recomienda validar el formato antes del Switch y clasificar montos no convertibles como datos inválidos, en lugar de provocar una caída operacional.

5.5. Casos límite

Se construyeron dos solicitudes adicionales.

Monto exactamente igual a U

Con `monto = 68000`, la regla `monto > 68000` es falsa. Por lo tanto, si no se activa otra causal, la solicitud queda aprobada.

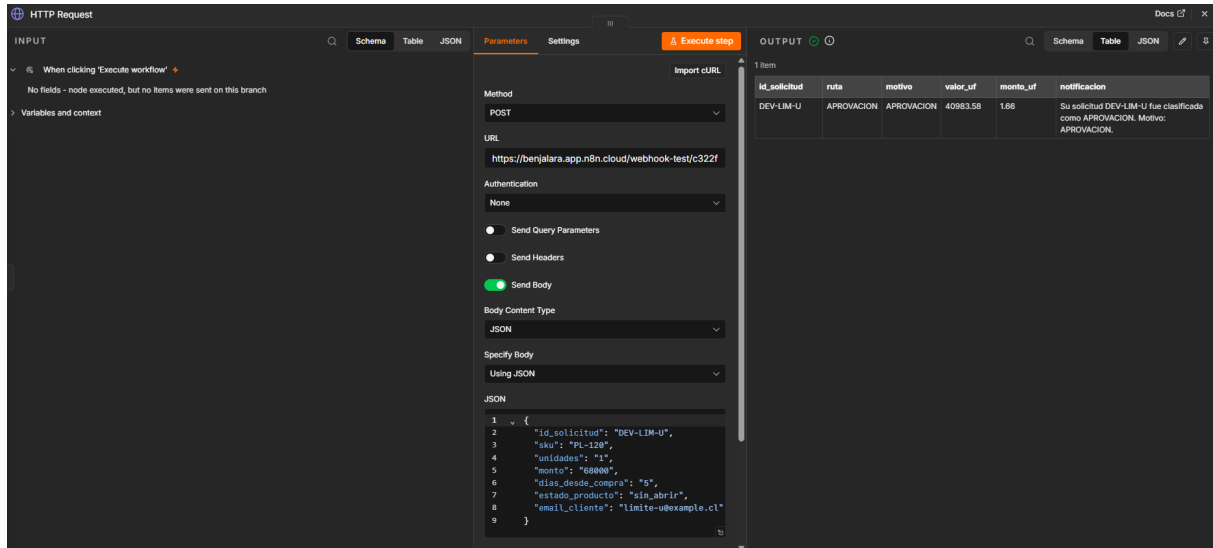


Figura 9: Caso límite $monto = U = \$68.000$, clasificado como APROBACION.

Días exactamente igual a D

Con `dias_desde_compra = 21`, la regla `dias > 21` es falsa. El día 21 se considera dentro del plazo.

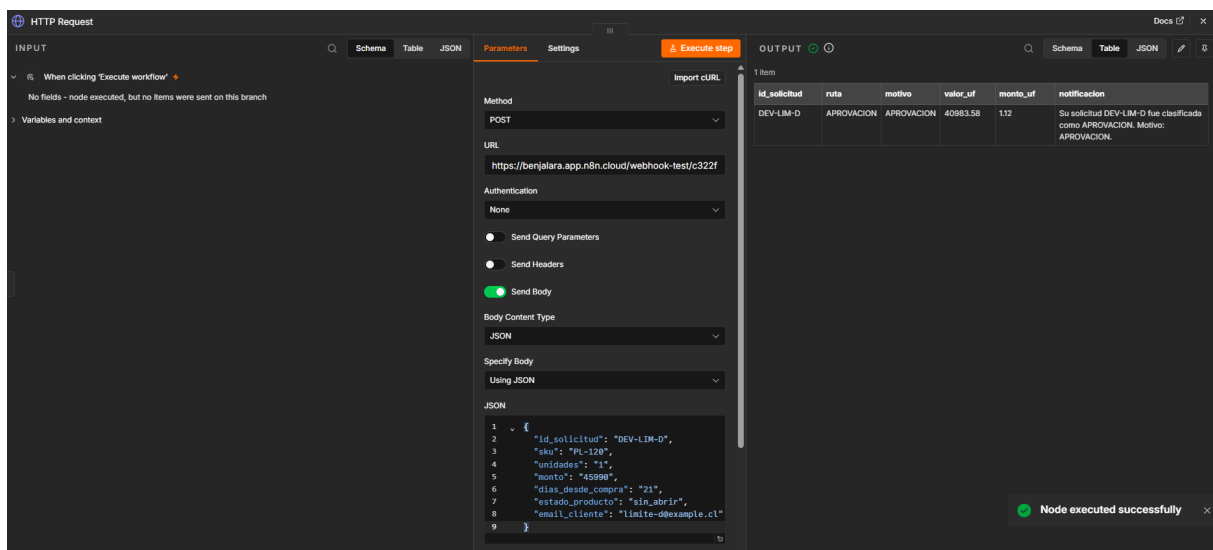


Figura 10: Caso límite $dias = D = 21$, clasificado como APROBACION.

5.6. Error de tipo a propósito

La comparación de monto se modificó temporalmente para reproducir una comparación textual. La versión utilizada de n8n no ofrece directamente el operador relacional “greater than” para String, por lo que se evaluó de forma explícita la comparación lexicográfica y se transformó su resultado a texto para la regla.

Con DEV-C01, el monto es "9990". Numéricamente:

$$9990 < 68000$$

Sin embargo, como texto se compara inicialmente "9" con "6", por lo que la condición textual puede evaluarse como verdadera. Como consecuencia, DEV-C01 fue enviado incorrectamente a la ruta de revisión. Esto demuestra que el tipo de dato afecta directamente el resultado de una comparación.

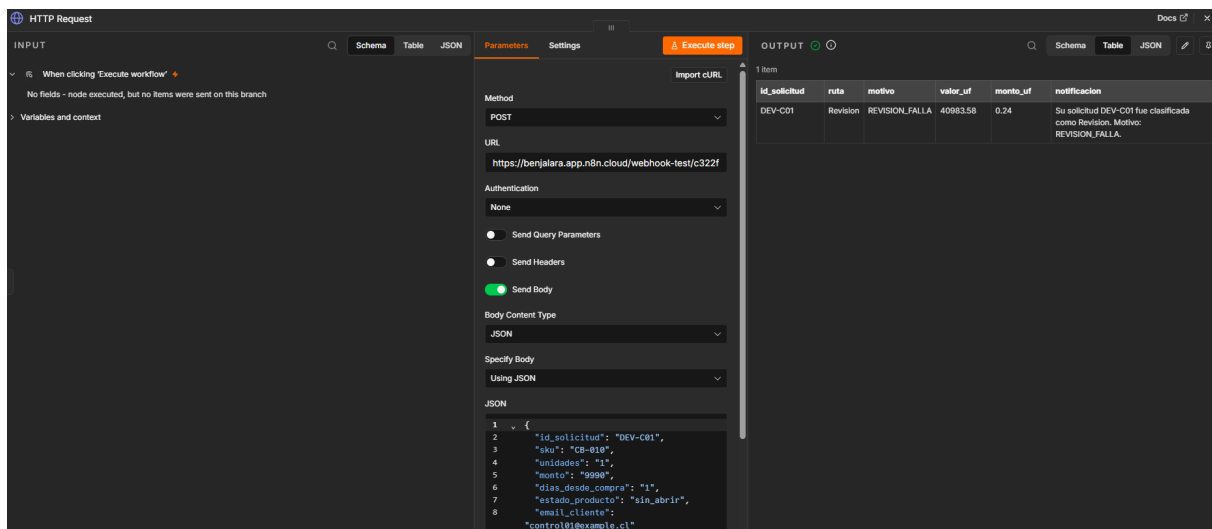


Figura 11: DEV-C01 enviado incorrectamente a revisión al forzar una comparación textual.

Después de la prueba, la regla se restituyó a tipo Number con `monto.toNumber() > 68000`.

5.7. Casos que el flujo no resuelve

Monto “veinte mil”

El flujo no puede clasificarlo porque las reglas suponen que el monto es convertible a número. Se requiere una etapa previa de validación de formato.

SKU “ZZ-999”

El flujo actual no consulta el catálogo para verificar la existencia del SKU. Por ese motivo, una solicitud con `sku = ZZ-999`, monto \$19.990, tres días y estado `sin_abrir` termina en APROBACION, aunque el SKU no existe en el catálogo entregado.

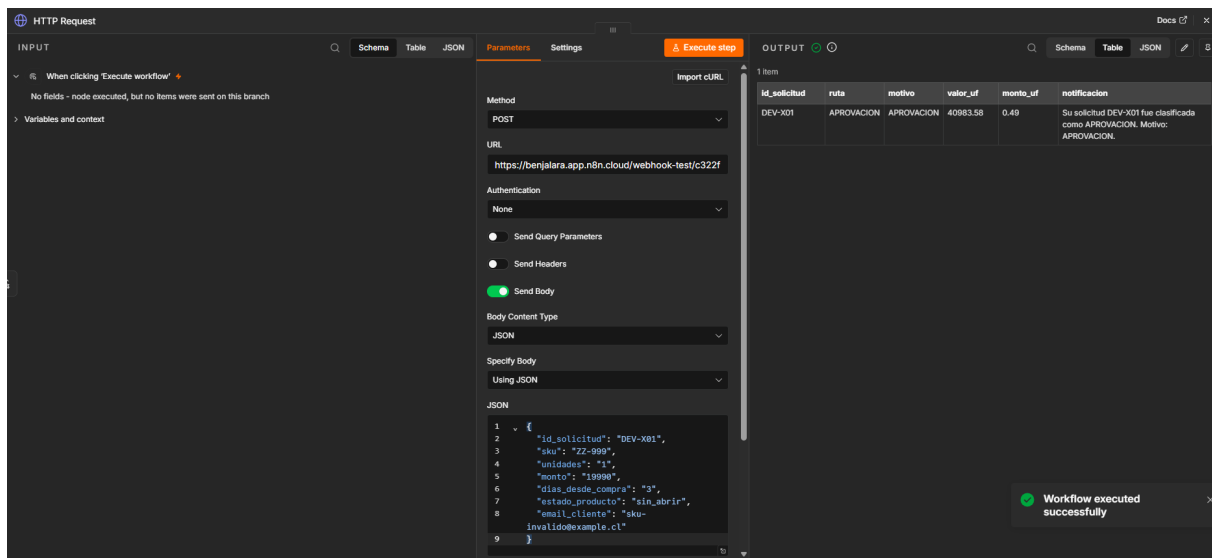


Figura 12: Solicitud con SKU inexistente clasificada como APROBACION por ausencia de validación de catálogo.

Para resolverlo se recomienda incorporar un lookup previo al catálogo y enviar los SKU inexistentes a datos inválidos o revisión.

6. Conclusiones y recomendación

El MVP demuestra que el triage inicial de devoluciones puede automatizarse de manera trazable y auditable con n8n. La separación entre Webhook, Switch, Edit Fields, HTTP Request, Data Table y Respond to Webhook hace visible la lógica de negocio y facilita revisar por qué una solicitud tomó una ruta determinada.

La precedencia de las reglas fue un aspecto central. Casos como un producto simultáneamente fuera de plazo y dañado por uso deben quedar asociados a la primera causal definida por la política. De igual forma, los casos límite confirmaron que $monto = U$ y $dias = D$ permanecen dentro de las condiciones de aprobación.

El principal riesgo observado se relaciona con la calidad y tipo de los datos de entrada. Los formularios entregan texto, por lo que una conversión incorrecta puede causar fallas o clasificaciones erróneas. Para una versión definitiva se recomienda incorporar una etapa de validación estructural antes del Switch, verificar SKU contra un catálogo persistente, utilizar un canal real de notificación, establecer monitoreo mediante Error Workflow y agregar controles para evitar registros duplicados.

En términos de mantenibilidad, la implementación sin código permite auditar visualmente cada regla. Dado el tamaño actual de la política, mantener el Switch como componente principal resulta adecuado. Si el número de reglas aumentara significativamente o se requirieran reglas parametrizadas por producto, canal o cliente, sería conveniente evaluar una separación entre configuración de reglas y lógica de ejecución.

7. Repositorio y reproducción

Repositorio público:

<https://github.com/benjaminlaraorellana2-creator/ICN292-Lab3-Lara-Benjamin.git>

Para reproducir la solución:

1. Importar `ICN292-Lab3-Lara-Benjamin-triage.json` en n8n.
2. Crear o verificar la Data Table `registro_devoluciones` con las columnas utilizadas por el flujo.
3. Publicar el workflow de triage y copiar su Production URL.
4. Importar `ICN292-Lab3-Lara-Benjamin-emisor.json` y configurar el HTTP Request con la URL del triage.
5. Ejecutar el workflow emisor para procesar las 15 solicitudes.
6. Importar `ICN292-Lab3-Lara-Benjamin-resumen.json`, verificar la zona horaria y publicar el workflow programado.
7. Confirmar que ningún JSON publicado contenga credenciales, tokens o contraseñas.

8. Evidencias incluidas

Las capturas utilizadas en este informe se adjuntan también como archivos independientes en la carpeta **evidencias**. Se incluyen ejecuciones exitosas, una ejecución fallida, resultados agregados, casos límite y pruebas de errores de tipo y validación.